

INDIVIDUAL MODULE-WISE LAB WORKBOOK

EXPERIMENT 2

Student Name: NISHA ANAND

Enrollment Number: 12023002022123

Department: CST

Institution: Institute of Engineering and Managent, Newtown

1. Module Name

Module 4 – Robot Framework

2. Experiment Title

Integrated Test Automation Using Robot Framework

3. Problem Statement

To study and apply Robot Framework for web test automation by creating test cases using basic syntax and keywords, variables, data-driven automation, custom keywords and libraries, assertions and verification, test setup and teardown, tags and test execution, and reports and logs. The module also covers working with browsers using Selenium Library, command-line execution, cloud execution using SauceLabs, scheduling through Jenkins, and organizing tests using user-defined keywords, resource files, and page-object-oriented structures.

4. Objective

The objective of this experiment is to develop an integrated understanding of Robot Framework and its use in automation testing. The experiment focuses on writing readable test cases, reusing keywords and variables, executing data-driven tests, performing assertions, managing setup and teardown, controlling test execution with tags, generating reports and logs, and integrating browser automation and execution tools.

5. Tools, Software and Concepts Used

- Robot Framework
- RIDE / suitable Robot Framework development environment
- Python
- Selenium Library
- Web browser
- Command Line
- PyCharm
- SauceLabs

- Jenkins
- Gherkin-style test organization
- Variables and data-driven automation
- BuiltIn library and custom Python keywords
- Assertions and verification
- Test setup and teardown
- Tags, reports and logs
- User-defined keywords and resource files
- Page Object Model concepts

6. Implementation Steps

1. Set up the Robot Framework environment and required libraries/tools for test automation.
2. Create a Robot Framework test case using basic syntax and keywords to open a web browser and navigate to a specified URL.
3. Use the Input Text keyword to enter data into a web form field and use Page Should Contain Element to verify the presence of an element.
4. Define variables such as usernames and passwords so that test data can be reused within test cases.
5. Implement data-driven automation by reading external test data and executing the same set of actions for multiple data sets.
6. Create custom Robot Framework keywords using Python for specific actions and use the BuiltIn library for string manipulation and mathematical operations.
7. Perform assertions and verification by comparing expected and actual results, including use of Should Be Equal As Strings for response validation.
8. Configure test setup and teardown so that common actions such as opening a browser, logging in, logging out, and cleaning resources can be performed around test cases.
9. Assign tags to test cases and suites and control execution using the --include and --exclude command-line options.
10. Execute tests in parallel using the --processes option where applicable.
11. Generate HTML reports containing test-case execution details and outcomes, and use the Log keyword to create custom messages for debugging.
12. Use Selenium Library for browser-based automation and interact with web applications through Robot Framework keywords.
13. Execute Robot Framework tests through the command line and use supported development environments such as PyCharm.
14. Use SauceLabs as a cloud-based execution environment for browser automation.

15. Use Jenkins to integrate and schedule automated test execution.
16. Compare procedural test organization with Gherkin-style organization and use user-defined keywords to improve readability and reuse.
17. Organize reusable functionality in resource files and separate page-related locators/UI methods from test assertions when applying a page-object-oriented structure.

7. Source Code

```
pip install robotframework
pip install robotframework-seleniumlibrary
pip install robotframework-ride
robot --version
ride.py
```

The environment should contain Python, Robot Framework, SeleniumLibrary, a compatible browser driver, and RIDE when graphical test creation is required.

```
*** Settings ***
Library      SeleniumLibrary
Suite Setup   Open Browser      https://example.com      chrome
Suite Teardown  Close All Browsers

*** Test Cases ***
Verify Example Page
    Maximize Browser Window
    Title Should Be      Example Domain
    [Teardown]          Capture Page Screenshot
```

Robot Framework uses a tabular syntax. Settings define libraries and lifecycle actions, while test cases contain keywords and arguments.

```
*** Keywords ***
Login With Valid Credentials
    [Arguments]    ${username}    ${password}
    Input Text      id=username    ${username}
    Input Password  id=password    ${password}
    Click Button    id=login

*** Test Cases ***
Login Test
    Login With Valid Credentials    testuser    secret
```

Keyword-driven testing expresses actions as reusable keywords. Data-driven testing runs the same workflow with different input values, which can be stored in variables, tables, external files, or resource files.

```
robot --outputdir results tests/  
robot -t "Verify Example Page" tests/  
robot --include smoke tests/  
robot --exclude regression tests/
```

Suite setup runs before the suite, test setup runs before each test, test teardown runs after each test, and suite teardown runs after the suite. These mechanisms support predictable resource preparation and cleanup.

```
*** Settings ***  
Library      SeleniumLibrary  
  
*** Test Cases ***  
Browser Interaction  
    Open Browser      https://example.com      chrome  
    Maximize Browser Window  
    Page Should Contain      Example Domain  
    Capture Page Screenshot  
    Close All Browsers
```

SeleniumLibrary provides keywords for opening browsers, locating elements, entering text, clicking controls, validating page content, taking screenshots, and closing browsers.

Robot and Pybot refer to Robot Framework execution commands in different versions or historical usage contexts. Current environments generally use the robot command.

- Run from PyCharm using a configured run configuration.
- Run from Command Prompt or terminal.
- Create a batch file containing the robot command.
- Use Windows Task Scheduler for scheduled execution.
- Run one test case, multiple suites, or tests selected by tags.
- Use output directories, variables, log levels, and include/exclude options.

A Jenkins job can execute Robot Framework tests as part of a CI/CD pipeline. The job may run manually, on a schedule, after source-control changes, or as part of another pipeline. Test results, logs, screenshots, and Robot Framework reports should be archived after each build.

```
robot --outputdir results tests/  
# Archive results, log.html, report.html and output.xml in Jenkins
```

Procedural scripts describe detailed actions in sequence. Gherkin-style descriptions use business-readable language such as Given, When, and Then. Robot Framework can support this readability through meaningful user-defined keywords.

```
*** Keywords ***
User Is On Login Page
    Go To      ${LOGIN_URL}

User Logs In With Valid Credentials
    Input Text    id=username    ${USERNAME}
    Input Password id=password    ${PASSWORD}
    Click Button  id=login

Dashboard Should Be Visible
    Page Should Contain    Dashboard
```

Resource files store shared keywords, variables, and locators so that multiple suites can reuse them. Page Object concepts separate page-specific locators and interactions from the test case layer, reducing duplication and simplifying maintenance when the UI changes.

```
*** Settings ***
Resource    resources/login_keywords.robot
Resource    resources/common_keywords.robot
```

8. Output Screenshots

Figure 1: Robot Framework Environment Setup

Python and Robot Framework installed successfully
Packages: robotframework, robotframework-seleniumlibrary
RIDE installed for graphical test development
Browser driver and environment variables configured

Observation: Illustrative expected output for this stage.

Figure 2: RIDE Simple Test Case

Figure 2: RIDE Simple Test Case

Test Case: Open Example Website
Keywords: Open Browser, Maximize Browser Window, Close All Browsers
Test case organized in a readable tabular format

Figure 3: Keyword and Data-Driven Test

Reusable user-defined keyword: Login With Valid Credentials

Test data supplied through variables or data tables

Multiple input combinations executed using the same keyword

Observation: Illustrative expected output for this stage.

Figure 4: Command Line and Setup/Teardown

Figure 4: Command Line and Setup/Teardown

robot --outputdir results tests/

Suite Setup: Open Application

Test Teardown: Capture Page Screenshot

Suite Teardown: Close All Browsers

Figure 5: Browser Automation Using SeleniumLibrary

Open Browser https://example.com chrome

Input Text id=username testuser

Click Button id=login

Page content and expected title validated

Observation: Illustrative expected output for this stage.

Figure 6: Script Running Options

Figure 6: Script Running Options

Robot command used from terminal or PyCharm

Single test: robot -t 'Test Name' tests/

Multiple suites: robot suite1.robot suite2.robot

Options include tags, variables, output directory and log level

Figure 7: Execution on Sauce Labs Browsers

Remote browser execution configured using Selenium Remote WebDriver
Browser, operating system and version selected as capabilities
Execution status and test results collected from remote run

Observation: Illustrative expected output for this stage.

Figure 8: Jenkins Scheduling and Execution

Figure 8: Jenkins Scheduling and Execution

Jenkins job configured with Robot Framework command
Build trigger: schedule / SCM / manual
Reports archived after execution
Failed builds marked using test result status

Figure 9: Readability with User-Defined Keywords

Procedural steps refactored into meaningful business keywords
Keywords moved to resource files for reuse
Setup and teardown maintained separately from test logic

Observation: Illustrative expected output for this stage.

Figure 10: Gherkin Style and Page Object Overview

Figure 10: Gherkin Style and Page Object Overview

Given the user is on the login page
When the user enters valid credentials
Then the dashboard should be displayed
Page Object concepts separate locators and actions from test cases

9. Result

The Robot Framework module was organized into one integrated automation workflow covering test-case creation, reusable keywords, variables, data-driven testing, verification, test lifecycle management, test selection, reporting, browser automation, and execution integration.

10. Observation

Robot Framework provides a keyword-driven approach that makes test cases readable and reusable. Variables and data-driven execution help separate test data from test actions, while setup/teardown, tags, reports, logs, custom keywords, and resource files support structured automation and maintenance.

11. Conclusion

This experiment provided an integrated understanding of Robot Framework for automation testing. The module covered the complete flow from writing basic browser test cases to organizing reusable automation components and managing execution, verification, reporting, and integration with supporting tools.